



SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
LAB TASKS

Course Code / Course Name : 10211CS213 / Python Programming
Year / Semester : 2025-2026/ SUMMER SEMESTER
Faculty Name : Dr. R. Thanga Selvi
Course Category : PROGRAM CORE
Slot : S2-1L14

Task 1: Running Python Script and various expressions in an interactive interpreter

a. Perform Basic Mathematical Computations

A billing software requires you to compute and display the results of operations between two price values entered by a user.

b. Evaluate Relational Expressions

A student grading system needs to compare two students' scores for ranking purposes.

c. Check Logical Conditions Across Multiple Inputs

You want to determine if a candidate qualifies in three skill tests using logical evaluation.

Task 2: Implement conditional, control and looping statements

a. Water Level Monitoring System (if-elif)

A dam monitoring system checks the water level and issues warnings. Write a Python program that takes water level (in meters) as input and prints:

- "Low Water Level" if $\text{level} < 50$
- "Normal Water Level" if $50 \leq \text{level} \leq 80$
- "High Water Level - Caution!" if $\text{level} > 80$

b. OTP Verification System (while loop)

An OTP verification system gives users 3 attempts to enter the correct OTP. Write a Python program that simulates asking for an OTP ("4567") up to 3 times until the correct OTP is entered or attempts are exhausted.

c. Multiplication Table Generator (for loop)

A teaching app needs to generate multiplication tables. Write a Python program that takes a number as input and prints its multiplication table up to 10 using a for loop.
(e.g., input: 3 → output: 3, 6, 9, ..., 30)

Task 3: Importing Python modules and packages in python programming

a. Movie Ticket Booking (List)

A cinema hall stores the ticket prices of different categories (Gold, Silver, Platinum) in a **list**. Write a program to:

- Calculate the total cost of tickets purchased by a customer.
- Find the highest-priced ticket.
- Find the lowest-priced ticket.

b. Employee Performance (Tuple)

A company stores each employee's name and their performance score in a **tuple**, e.g., ("Arun", 85).

Write a program to:

- Store data for 5 employees.
- Display the employee with the highest score.
- Display all employees who scored more than 80.

c. Library Book Tracker (Dictionary)

A library maintains a **dictionary** of books and their authors. Example:
{ "Python Basics": "Guido", "AI Fundamentals": "John" }

Write a program to:

- Add a new book-author pair entered by the user.
- Search for the author of a given book.
- Display all book-author pairs in alphabetical order of book name.

Task 4. Use various data types, List, Tuples and Dictionary in python programming

a. Student Marks Manager

A teacher wants to manage marks of students.
Write a Python program to:

1. Store marks of students in a **list**.
2. Find the **highest and lowest marks**.
3. Calculate the **average mark**.
4. Sort the marks in **ascending order**.

b. Employee Records

A company stores employee details in a **tuple** (ID, Name, Salary).
Write a Python program to:

1. Store records of 5 employees.
2. Display the details of the **highest paid employee**.
3. Extract only the **employee names** into a list.
4. Count how many employees earn more than 50,000.

c. Shopping Cart

An online store manages a **shopping cart** using a dictionary, where:

- **Key = Product Name**
- **Value = Price**

Write a Python program to:

1. Add at least 5 products to the dictionary.
2. Display all products with prices.
3. Find the **total bill amount**.
4. Update the price of one product.
5. Remove a product from the cart.

Task - 5.Implement various Searching and Sorting Operations in python programming

a. Student Record Search

A school maintains a list of students with their roll numbers.
Write a Python program to:

1. Store roll numbers in a list.
2. Implement **Linear Search** to find whether a roll number exists.
3. Implement **Binary Search** to locate a student's roll number (after sorting).
4. Display appropriate messages if found or not.

b. Product price sorting

An online shopping portal stores prices of items.
Write a Python program to:

1. Store product prices in a list.
2. Implement the following sorting algorithms:
 - **Bubble Sort** (ascending order).
 - **Selection Sort** (descending order).
 - **Insertion Sort** (ascending order).
3. Display the sorted list after each sorting technique.

Task 6.Implement various text file operation

a. Student Record File Handling

A school wants to maintain student records in a text file.
Write a Python program to:

1. **Create** a text file named students.txt and store student names with their marks.
2. **Read** and display all student records.
3. **Append** new student data to the same file.
4. **Search** for a student by name in the file.

b. Word Count Analyzer

A publishing company wants to analyze the content of text documents.
Write a Python program to:

1. **Read** a text file given by the user.
2. Count and display the number of:
 - Lines
 - Words
 - Characters
3. Display the **frequency of each word** in the file.
4. Store the analysis result in a new file named analysis.txt.

Task No: 7 Utilizing ‘Functions’ concepts in Python Programming.

a. Simple Banking System

Create a Python program using functions to:

- Add money to a user’s account (Deposit).
- Deduct money from the account (Withdraw), only if there is enough balance.
- Display the available account balance.

b. Student Performance Evaluator

Design a Python program using functions to:

- Take marks for three different subjects as input.
- Compute the total score, average marks, and assign a grade (A/B/C/Fail) based on performance.
- Show the final result by calling a separate function.

Task 8 Implement python generator and decorators

a. Fibonacci Number Generator

Develop a Python program that:

- Defines a generator function to produce Fibonacci numbers up to a given limit n.
- Prints the sequence of Fibonacci numbers as specified by the user’s input.

b. Measuring Function Execution Time with a Decorator

Construct a Python program that:

- Creates a decorator to compute and display how long a function takes to run.
- Uses this decorator on a function that performs sorting on a list of randomly generated numbers.

Task 9 Implement Exceptions and Exceptional handling in Python.

a. Division Error Handling

Write a Python program that accepts two numbers from the user and performs division.

- Use try-except-finally blocks to handle the **ZeroDivisionError** when the denominator is zero.
- Display an appropriate error message and ensure the finally block is executed.

b. Custom Exception for Age Validation

Write a Python program to validate a person's age for voting eligibility.

- Create a **user-defined exception** called **InvalidAgeError**.
- Raise this exception if the entered age is less than 18, otherwise display "Eligible to Vote."

Task 10 Use Matplotlib module for plotting in python

A company wants to analyze the sales performance of its products using charts. Write a Python program using Matplotlib to:

1. Plot a **line chart** showing monthly sales (in units) of 3 products across 6 months.
2. Plot a **bar chart** comparing the total sales of each product.
3. Plot a **pie chart** showing the market share (percentage of sales) of the 3 products in the last month.
4. Add suitable **titles, axis labels, legends, and grid lines** to make the graphs informative.

Task 11 Use Tkinter module for UI design

Employee Registration Form using Tkinter

Design a simple GUI application using Tkinter to collect and display employee details.

1. Create a form with input fields for **Employee ID, Name, Department, and Contact Number**.
2. Add **Submit** and **Reset** buttons.
 - On clicking **Submit**, the entered details should be shown in a text area or a message box and On clicking **Reset**, all input fields should be cleared.
3. Use appropriate **labels, entry widgets, and buttons**.
4. Improve the interface by adding a **window title, fixed geometry, and background color**.

Task 12 Simulate Gaming concepts using Pygame

Car Dodging Game using Pygame

Develop a simple Pygame application to demonstrate basic gaming elements.

1. Create a game window with a **title** and a **background color**.
2. Display a **car object** (rectangle or image) that can move left and right using arrow keys.
3. Add **falling obstacles** (enemy objects) that move downwards automatically.
4. Detect collision between the car and any obstacle:
 - If a collision occurs, display **“Game Over”** and end the game.
5. Implement **event handling, continuous screen updates, and frame rate control**.

Use Cases:

Use Case 1: Finding the Winning Strategy in a Card Game using python.

Use Case 2: Develop an application to perform Plotting Probabilities Using Matplotlib

Use Case 3: Design an application for Simulating Random Coin-Flips and Dice-Rolls Using NumPy